TEXT-BASED INVOICE DATA EXTRACTION

Python Activity Record

1. Problem Statement

Develop a Python program that reads one invoice record in the format CODE|QUANTITY|AMOUNT, separates the three fields, checks that the record has the expected number of fields, converts quantity and amount to numeric values, validates the values, and displays the product code, quantity, and amount rounded to two decimal places. The program should show suitable messages when the record is malformed or contains invalid values.

2. Objectives

- Read invoice information from the keyboard.
- Split a text record using the pipe (|) separator.
- Check that the record contains exactly three fields.
- Convert quantity to an integer and amount to a floating-point number.
- Validate that the product code is not empty, quantity is greater than zero, and amount is not negative.
- Handle non-numeric quantity or amount using ValueError exception handling.
- Display the accepted invoice details with the amount rounded to two decimal places.

3. Assumptions

- The user enters one invoice record in the order CODE|QUANTITY|AMOUNT.
- The pipe symbol (|) is the required separator.
- The product code is text; quantity is a whole number; amount is a numeric value.
- Quantity must be greater than zero, and amount must be zero or more.
- The program checks the format and basic values only; it does not read a file, save records, or verify an invoice against a database.
- The program rounds the displayed amount to two decimal places but does not change the original stored value.

4. Algorithm

1. Start the program.

2. Ask the user to enter an invoice record in CODE|QUANTITY|AMOUNT format.

3. Split the record at each pipe symbol.

4. If the number of fields is not three, display the malformed-record message and stop processing.

5. Remove leading and trailing spaces from the product code.

6. Convert quantity to an integer and amount to a floating-point number.

7. If conversion fails, display the numeric-input error message.

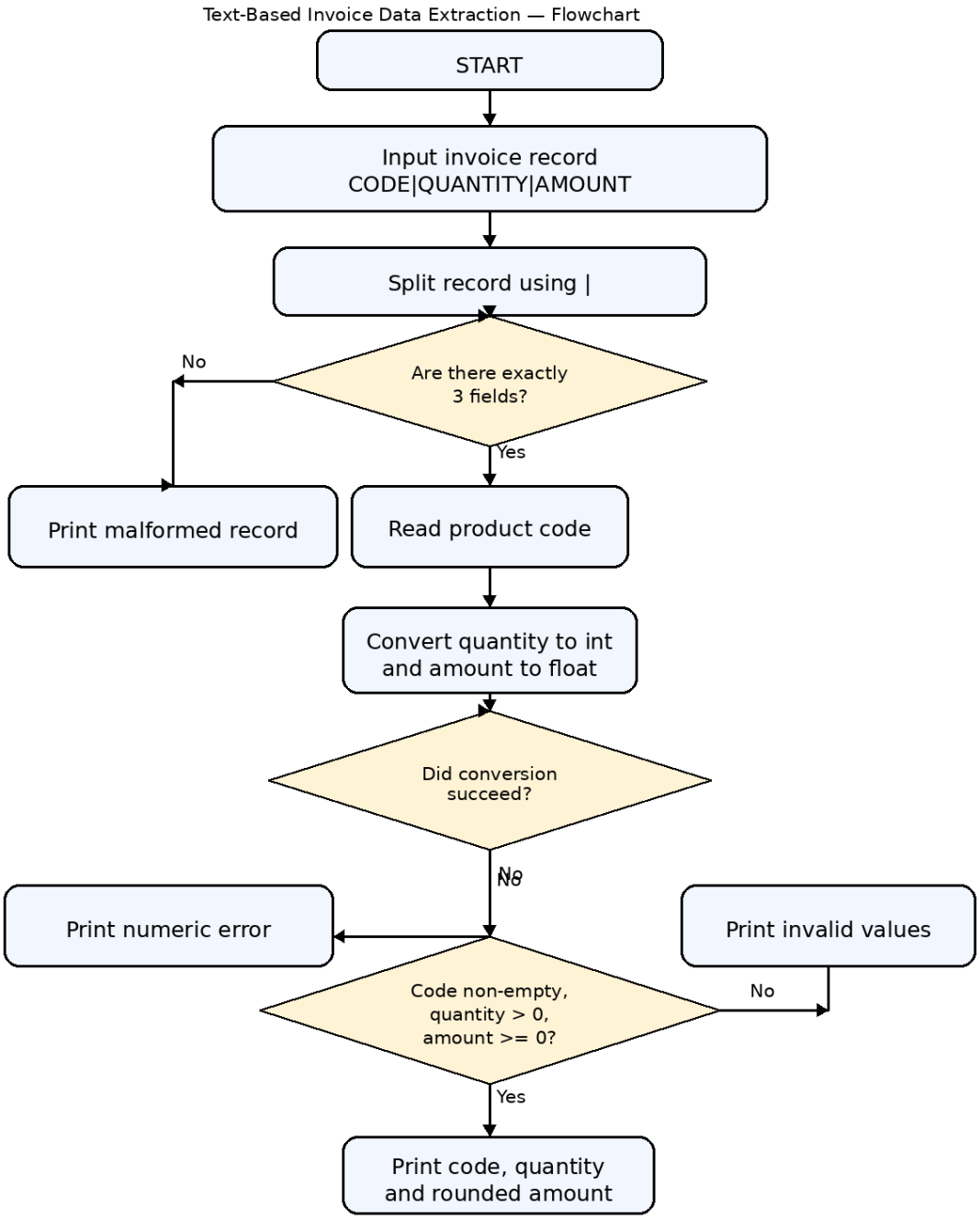
8. If the code is empty, quantity is not positive, or amount is negative, display the invalid-values message.

9. Otherwise, display the product code, quantity, and amount rounded to two decimal places.

10. End the program.

5. Flowchart

The flowchart below shows the main decisions and output paths.



6. Pseudocode

```
BEGIN
  INPUT record
  fields = SPLIT record USING "|"

  IF LENGTH(fields) != 3 THEN
    DISPLAY "Malformed record: expected 3 fields."
  ELSE
    code = TRIM(fields[0])
    TRY
      quantity = INTEGER(fields[1])
      amount = FLOAT(fields[2])

      IF code is empty OR quantity <= 0 OR amount < 0 THEN
        DISPLAY "Invalid code, quantity or amount."
      ELSE
        DISPLAY product code
        DISPLAY quantity
        DISPLAY amount rounded to 2 decimal places
      END IF
    CATCH ValueError
      DISPLAY "Quantity and amount must be numeric."
    END TRY
  END IF
END
```

7. Python Source Code

```
record = input("Invoice record (CODE|QUANTITY|AMOUNT): ")

fields = record.split("|")

if len(fields) != 3:
    print("Malformed record: expected 3 fields.")
else:
    code = fields[0].strip()
    try:
        quantity = int(fields[1])
        amount = float(fields[2])
        if not code or quantity <= 0 or amount < 0:
            print("Invalid code, quantity or amount.")
        else:
            print("Product code:", code)
            print("Quantity:", quantity)
            print("Amount: Rs.", round(amount, 2))
    except ValueError:
        print("Quantity and amount must be numeric.")
```

8. Test Cases

Test	Input	Expected result	Purpose
1	INV101|3|1250.567	Shows INV101, quantity 3, amount Rs. 1250.57	Valid record

2	INV101|3	Malformed record: expected 3 fields.	Missing field
3	INV102|0|450	Invalid code, quantity or amount.	Quantity is zero
4	INV103|three|450	Quantity and amount must be numeric.	Non-numeric quantity
5	|2|100	Invalid code, quantity or amount.	Empty product code
6	INV104|2|-10	Invalid code, quantity or amount.	Negative amount

9. Output Screenshots

The following are sample console outputs based on the program logic.

9.1 Valid invoice record

Sample console output – valid record

```
Invoice record (CODE|QUANTITY|AMOUNT): INV101|3|1250.567
Product code: INV101
Quantity: 3
Amount: Rs. 1250.57
```

9.2 Malformed record

Sample console output – malformed record

```
Invoice record (CODE|QUANTITY|AMOUNT): INV101|3
Malformed record: expected 3 fields.
```

9.3 Invalid values

Sample console output – invalid values

```
Invoice record (CODE|QUANTITY|AMOUNT): INV102|0|450
Invalid code, quantity or amount.
```

9.4 Non-numeric quantity

Sample console output – non-numeric input

```
Invoice record (CODE|QUANTITY|AMOUNT): INV103|three|450
Quantity and amount must be numeric.
```

10. Conclusion

The program demonstrates basic text-based invoice data extraction using Python. It splits a record into fields, converts numeric values, checks simple validation rules, handles conversion errors, and prints a formatted result. It is suitable for learning string processing, conditional statements, type conversion, and exception handling. Its validation is basic and does not confirm whether the invoice or product code exists in a real business system.